

Observation task: 03

Date: 15/09/2026

Title: UPI Wallet Payment System using Interface, Classes, Objects and Exception Handling

Objective / Aim

To design and implement a **UPI Wallet Payment System** in Java using **interfaces, classes, objects, encapsulation, exception handling, and String operations**, demonstrating wallet creation, adding money, UPI payment, balance checking, UPI validation, amount validation, and insufficient balance handling.

System Requirements

Hardware

- Standard PC/Laptop
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
 - Java Development Kit (JDK 17 or later)
 - Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
 - Command Prompt / Terminal
-

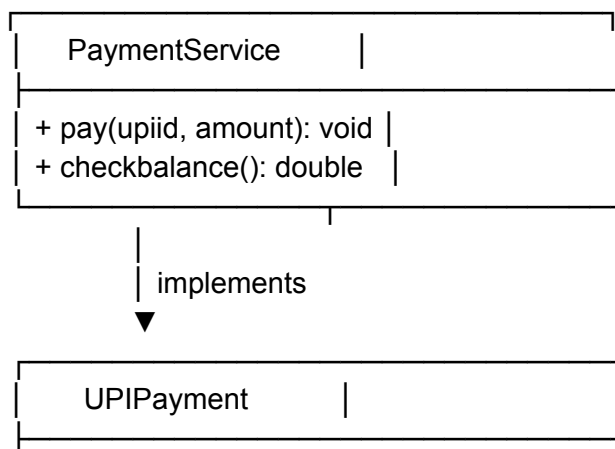
Algorithm / Logic Description

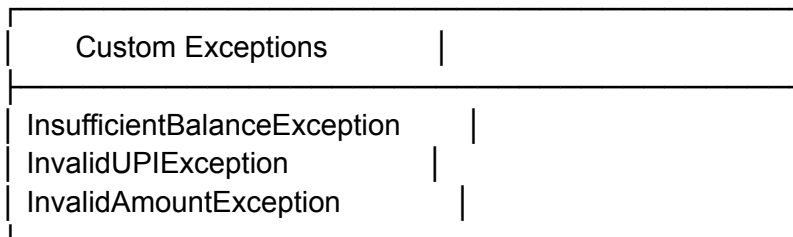
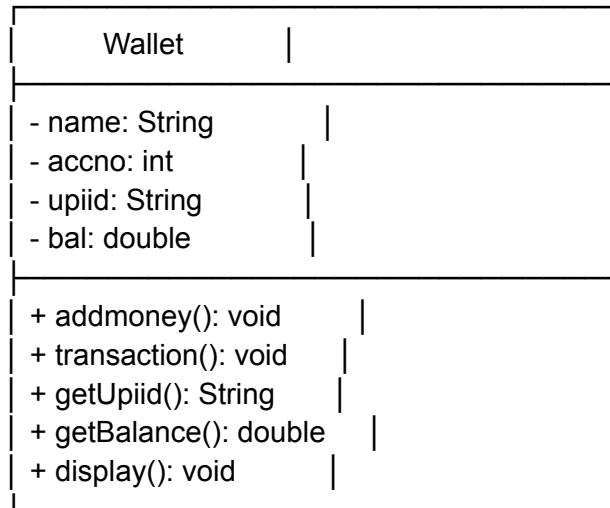
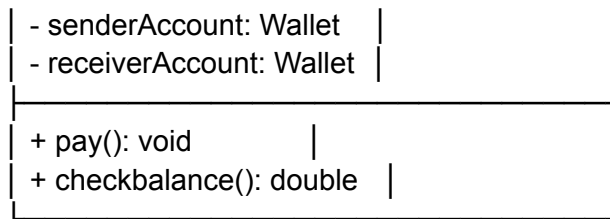
1. Create an interface named `PaymentService`.
2. Define the methods:
 - `pay()` — performs UPI payment.
 - `checkbalance()` — returns the available balance.
3. Create custom exceptions:

- `InsufficientBalanceException`
 - `InvalidUPIException`
 - `InvalidAmountException`
4. Create a `Wallet` class with private attributes:
 - Name
 - Account number
 - UPI ID
 - Balance
 5. Create a constructor in the `Wallet` class to initialize wallet details.
 6. Define methods in the `Wallet` class:
 - `addmoney()` — adds money to the wallet.
 - `transaction()` — deducts money from the wallet.
 - `getUpiid()` — returns the UPI ID.
 - `getBalance()` — returns the current balance.
 - `display()` — displays wallet details.
 7. Create a `UPIPayment` class that implements the `PaymentService` interface.
 8. Store the sender and receiver wallet objects in the `UPIPayment` class.
 9. Validate the receiver UPI ID using the `contains()` and `equals()` String methods.
 10. Check whether the payment amount is greater than zero.
 11. Check whether the sender has sufficient balance.
 12. If all conditions are valid, deduct money from the sender and add it to the receiver.
 13. Handle exceptions using `try`, `catch`, and `finally`.
 14. Create sender and receiver wallet objects in the `main()` method.
 15. Perform the payment and display the final wallet details.
-



UML Class Diagram





Relationship: **UPIPayment** implements the **PaymentService** interface and uses **Wallet** objects for sender and receiver accounts.



Source Code

```
import java.util.Scanner;
```

```
// Payment Service Interface
```

```

interface PaymentService {
    void pay(String upiid, double amount);
    double checkbalance();
}

// Custom Exception for insufficient balance
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception for invalid UPI
class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception for invalid amount
class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet Class
class Wallet {

    private String name;
    private int accno;
    private String upiid;
    private double bal;

    // Constructor
    Wallet(String name, int accno, String upiid, double bal) {
        this.name = name;
        this.accno = accno;
        this.upiid = upiid;
        this.bal = bal;
    }

    // Add money to wallet
    void addmoney(double amount) {

```

```

        bal = bal + amount;
    }

    // Deduct money from wallet
    void transaction(double amount) {
        bal = bal - amount;
    }

    // Get UPI ID
    String getUpiid() {
        return upiid;
    }

    // Get current balance
    double getBalance() {
        return bal;
    }

    // Display wallet details
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Account Number: " + accno);
        System.out.println("UPI ID: " + upiid);
        System.out.println("Balance: Rs." + bal);
    }
}

// UPI Payment Class
class UPIPayment implements PaymentService {

    private Wallet senderAccount;
    private Wallet receiverAccount;

    // Constructor
    UPIPayment(Wallet sender, Wallet receiver) {
        senderAccount = sender;
        receiverAccount = receiver;
    }

    // Perform UPI payment
    public void pay(String upiid, double amount) {

        try {

```

```

// Validate UPI format
if (!upiId.contains("@")) {
    throw new InvalidUPIException("Invalid UPI ID");
}

// Check receiver UPI ID
if (!receiverAccount.getUpiId().equals(upiId)) {
    throw new InvalidUPIException(
        "Receiver UPI ID not found"
    );
}

// Validate payment amount
if (amount <= 0) {
    throw new InvalidAmountException(
        "Amount must be greater than zero"
    );
}

// Check sender balance
if (amount > senderAccount.getBalance()) {
    throw new InsufficientBalanceException(
        "Insufficient balance"
    );
}

// Transfer money
senderAccount.transaction(amount);
receiverAccount.addmoney(amount);

System.out.println("\nPayment Successful");
System.out.println("From: " + senderAccount.getUpiId());
System.out.println("To: " + receiverAccount.getUpiId());
System.out.println("Amount: Rs." + amount);
}

// Handle invalid UPI exception
catch (InvalidUPIException e) {
    System.out.println("Payment Failed: " + e.getMessage());
}

// Handle invalid amount exception
catch (InvalidAmountException e) {
    System.out.println("Payment Failed: " + e.getMessage());
}

```

```

    }

    // Handle insufficient balance exception
    catch (InsufficientBalanceException e) {
        System.out.println("Payment Failed: " + e.getMessage());
    }

    // Execute after transaction
    finally {
        System.out.println("Transaction Completed");
    }
}

// Check sender balance
public double checkbalance() {
    return senderAccount.getBalance();
}
}

```

// Main Class

```

public class TaskIII {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Creating sender wallet object
        Wallet naveenAccount = new Wallet(
            "Naveen",
            101,
            "naveen@upi",
            5000
        );

        // Creating receiver wallet object
        Wallet rahulAccount = new Wallet(
            "Rahul",
            102,
            "rahul@upi",
            2000
        );

        // Creating UPI payment object
        UPIPayment paymentSystem =

```

```

        new UPIPayment(naveenAccount, rahulAccount);

System.out.println("----- Initial Wallet Details -----");

naveenAccount.display();

System.out.println();

rahulAccount.display();

// Taking payment details from user
System.out.print("\nEnter Receiver UPI ID: ");
String receiverUPI = sc.nextLine();

System.out.print("Enter Amount: ");
double amount = sc.nextDouble();

// Performing payment
paymentSystem.pay(receiverUPI, amount);

System.out.println("\n----- Final Wallet Details -----");

naveenAccount.display();

System.out.println();

rahulAccount.display();

// Display final sender balance
System.out.println(
    "\nNaveen Available Balance: Rs." +
    paymentSystem.checkbalance()
);

sc.close();
}
}

```



Compilation & Execution Commands

Save the source code exactly as:

TaskIII.java

Compile the program:

```
javac TaskIII.java
```

Run the compiled program:

```
java TaskIII
```



Output Screen

Example: Successful Payment

----- Initial Wallet Details -----

Name: Naveen

Account Number: 101

UPI ID: naveen@upi

Balance: Rs.5000.0

Name: Rahul

Account Number: 102

UPI ID: rahul@upi

Balance: Rs.2000.0

Enter Receiver UPI ID: rahul@upi

Enter Amount: 1500

Payment Successful

From: naveen@upi

To: rahul@upi

Amount: Rs.1500.0

Transaction Completed

----- Final Wallet Details -----

Name: Naveen

Account Number: 101

UPI ID: naveen@upi

Balance: Rs.3500.0

Name: Rahul

Account Number: 102

UPI ID: rahul@upi

Balance: Rs.3500.0

Naveen Available Balance: Rs.3500.0

Example: Invalid UPI

Enter Receiver UPI ID: rahulupi

Enter Amount: 1500

Payment Failed: Invalid UPI ID

Transaction Completed

Example: Insufficient Balance

Enter Receiver UPI ID: rahul@upi

Enter Amount: 6000

Payment Failed: Insufficient balance

Transaction Completed

Example: Invalid Amount

Enter Receiver UPI ID: rahul@upi

Enter Amount: -500

Payment Failed: Amount must be greater than zero

Transaction Completed

```
Account Number: 101
UPI ID: naveen@upi
Balance: ₹5000.0

Name: Rahul
Account Number: 102
UPI ID: rahul@upi
Balance: ₹2000.0

Enter Receiver UPI ID: rahul@upi
Enter Amount: 1000

Payment Successful
From: naveen@upi
To: rahul@upi
Amount: ₹1000.0
Transaction Completed

----- Final Wallet Details -----
Name: Naveen
Account Number: 101
UPI ID: naveen@upi
Balance: ₹4000.0

Name: Rahul
Account Number: 102
UPI ID: rahul@upi
Balance: ₹3000.0

Naveen Available Balance: ₹4000.0
```

```
PS C:\Users\navee\OneDrive\Desktop\programs> javac TaskIII.java
PS C:\Users\navee\OneDrive\Desktop\programs> java TaskIII
----- Initial Wallet Details -----
Name: Naveen
Account Number: 101
UPI ID: naveen@upi
Balance: ₹5000.0

Name: Rahul
Account Number: 102
UPI ID: rahul@upi
Balance: ₹2000.0

Enter Receiver UPI ID: rahul@123
Enter Amount: 1000
Payment Failed: Receiver UPI ID not found
Transaction Completed

----- Final Wallet Details -----
Name: Naveen
Account Number: 101
UPI ID: naveen@upi
Balance: ₹5000.0

Name: Rahul
Account Number: 102
UPI ID: rahul@upi
Balance: ₹2000.0

Naveen Available Balance: ₹5000.0
```